MACHINE LEARNING
MODEL DEVELOPMENT

August 5, 2026 | 17 min read

What Is Machine Learning Model Training?

David Weedmark

← RETURN TO BLOG HOME

DOMINO PLATFORM

The enterprise platform to build, deliver, and govern AI

Watch the 15 minute on-demand demo to get an overview of the Domino Enterprise AI Platform.

WATCH DEMO

What is machine learning model training?

Machine learning (ML) model training is the process of feeding an algorithm enough examples of a phenomenon that it can find patterns in that data on its own, and then use those patterns to make predictions on data it has never seen before. It is the step where a ML algorithm becomes a working model, a set of learned parameters that turns new inputs into useful outputs, whether that output is a fraud score, a diagnosis probability, or the next token in a sentence.

How machine learning model training works

Model training is a search problem. A data science team selects an algorithm, then adjusts its internal parameters, such as weights and biases, until the algorithm's predictions come as close as possible to the true answers in the training data. That closeness is measured by a loss function, a formula that shows how wrong a prediction is. Training a ML model means repeatedly nudging the parameters to reduce that loss, one batch of data at a time.

The mechanics differ depending on the learning technique. In supervised learning, the model learns a mapping between input features and a known target label like a continuous number indicating yield or a categorical label like Good Credit. In unsupervised learning, there is no label to check against, so the model instead learns structure among the data features themselves, such as which records cluster together. A newer category has become just as relevant to this definition: fine-tuning. When a team adapts a pretrained foundation model to a narrower task, they are still training a machine learning model. They are simply starting from a model that already learned general patterns from a massive corpus, rather than starting from randomly initialized weights.

Why model training in machine learning matters

A trained model is only as good as the process that produced it. Two teams can use the identical algorithm and get wildly different results depending on how carefully they collected data, split it into training and testing samples, and validated the outcome. Weak training produces a model that looks accurate on paper but fails the moment it meets real customers, real transactions, or real patients.

This is also where most of the cost sits. Data collection, compute time, and the data science team's hours are concentrated in training, not in the eventual deployment. Getting training right the first time is usually the difference between a project that ships on schedule and one that gets rebuilt twice.

Supervised vs unsupervised model training

Most model training falls into one of two camps, and the difference determines everything downstream, from data preparation to how you judge model performance.

Supervised learning trains a model on labeled data: each training example includes both the input features and the correct answer. A spam classifier trained on emails marked "spam" or "not spam" is a supervised learning example. Because the ground truth is known, supervised training can use direct metrics like accuracy or mean squared error to judge model performance.

Unsupervised learning trains a model without labels. The algorithm looks for structure on its own, grouping similar records together, called clustering, or reducing many features down to the handful that carry the most information, called dimensionality reduction. There is no single correct answer to check against, so evaluation depends more on domain judgment: does the resulting grouping make business sense?

A third pattern sits alongside these two and has become central to generative AI. This is self-supervised pretraining followed by supervised fine-tuning. Large language models are first trained on unlabeled text at a scale no human-labeled dataset could match, learning general language patterns without any explicit labels. Teams then fine-tune that pretrained model on a smaller, labeled dataset specific to their task, whether that is summarizing contracts or answering customer support questions. The mechanics, a loss function and gradient-based updates, are the same ones described above. What changes is the starting point.

How to train a machine learning model: the ML model training process

Training a model is a systematic, repeatable process, not a single step. Skipping ahead to "train the model" without doing the groundwork first is the most common reason a project stalls in review.

Define the problem and success criteria

Before touching data, agree on what the model needs to predict and how you will know if it succeeded. A fraud model optimized purely for accuracy can still fail in production if it misses the 2% of transactions that represent the actual fraud, so the success metric has to reflect the real business cost of different kinds of errors.

Collect and prepare training data

Data collection is where training quality is won or lost. The training data needs to represent the conditions the model will actually see in production, including edge cases, seasonal patterns, and underrepresented groups. Incomplete or biased training data produces a model that inherits those gaps, no matter how sophisticated the algorithm.

Feature engineering and preprocessing

Raw data rarely arrives in a form an algorithm can use directly. Feature engineering turns raw fields into inputs a model can learn from, whether that means encoding categorical variables, scaling numeric ranges, or extracting a day-of-week feature from a timestamp. Preprocessing also includes data quality issues such as handling missing values and removing duplicate or corrupted records before they distort training.

Split data into training, validation, and test sets

A model cannot be fairly evaluated on the same data it learned from. Splitting the dataset into a training set, a validation set, and a test set lets a team train on one portion, tune decisions on a second portion, and get an honest read on unseen data with the third. A common approach is k-fold cross-validation, which splits the data into k folds, trains on k-1 of them, and rotates the held-out fold until every fold has served as the test set once. Averaging the scores across folds gives a more stable performance estimate than a single train-test split, particularly on smaller datasets.

Select an algorithm

There is no universally best machine learning algorithm. The right choice depends on the size of the dataset, how interpretable the output needs to be, and how much latency the application can tolerate. Comparing several ML algorithms against the same training and testing sets before committing to one is standard practice, since performance on paper rarely tells the whole story. A regulated lending decision may call for a simpler, more explainable model even if a more complex one scores higher, because the interpretability requirement outweighs an accuracy gain.

Train the model

This is the step most people picture when they hear "model training". The algorithm processes the training data, computes a loss for its current predictions, and uses an optimization method such as gradient descent to adjust its parameters in the direction that reduces that loss. The process repeats over many passes through the data, called epochs, until the loss stops improving meaningfully.

Evaluate the model

Once trained, the model is tested against the held-out test set to see how it performs on data it never saw during training. Metrics vary by problem type including precision and recall for classification and root mean squared error for regression. The goal is the same: confirm the model generalizes on unseen data rather than having simply memorized the training set.

Tune hyperparameters

Hyperparameters are the settings a team chooses before training starts, such as the learning rate, the number of trees in a random forest, or the strength of a regularization penalty. Unlike the model's internal parameters, hyperparameters are not learned from data directly. Teams typically search across a range of hyperparameter combinations, retraining and re-evaluating for each one, to find the configuration that produces the best validation performance.

Common challenges in ML model training

Overfitting and underfitting

A model that overfits has effectively memorized the training data, including its noise, and performs poorly on anything new. A model that underfits has not learned enough structure to be useful even on the training data itself. The fix for overfitting usually involves more training data, regularization, or a simpler model. The fix for underfitting usually means a more expressive model or better features. Cross-validation is one of the most reliable ways to catch overfitting before it reaches production.

Data quality and compute cost

Dirty, mislabeled, or unrepresentative data caps how good any trained ML model can become, and no amount of algorithm tuning fixes a labeling problem. Compute cost has also become a bigger factor than it was even a few years ago. Training a gradient boosted tree on a few million rows with a modest feature set might finish on a laptop. Fine-tuning a foundation model, or running the reinforcement learning loops that teach an agentic AI system to choose the right sequence of actions, can require GPU clusters running for days, and larger jobs increasingly need to run across HPC infrastructure rather than a single node. Teams now weigh accuracy gains against compute cost explicitly, rather than treating compute as a fixed cost of doing business.

From training to production: retraining at scale

Training is not a one-time event. Data drifts. Customer behavior shifts, new fraud patterns emerge, and a model trained on last year's data slowly loses relevance. Most production systems need a retraining cadence, whether that means retraining monthly on a rolling window of new data or triggering retraining automatically when monitored performance drops below a specified threshold.

Generative and agentic AI systems raise the stakes on this problem rather than removing it. An agentic AI system that plans multi-step tasks generates a constant stream of new interaction data: which tool calls succeeded, where a plan needed correction, which outputs a human reviewer rejected. That feedback becomes training data in its own right, used to fine-tune the underlying model or a separate reward model that guides the agent's decisions. Retraining at scale is no longer just about keeping a fraud model current. It now also covers keeping a fine-tuned LLM or an agent's policy aligned with how it is actually being used.

Machine learning model training with Domino

Training a model reliably at scale requires more than a good algorithm. It requires infrastructure that gives a data science team fast, self-service access to the compute a job actually needs, whether that is a CPU cluster for a gradient boosting run, a multi-GPU environment for fine-tuning a large language model, or an existing HPC cluster for the largest distributed training jobs. It also requires an environment that can reproduce exactly how a model was trained months later, since a model that cannot be reproduced cannot be trusted, audited, or safely retrained.

Domino's Enterprise AI platform gives teams a single place to run training jobs on flexible, on-demand infrastructure, including Slurm-based HPC clusters for teams that already run one, and track every experiment automatically while managing the full lifecycle from training through validation and deployment. That same foundation extends to newer training workloads, including fine-tuning foundation models and building the evaluation loops that agentic AI systems need to improve safely over time. For teams building Agentic AI applications alongside traditional ML models, having both on one governed platform keeps training decisions, whether for a classic supervised model or a fine-tuned LLM, documented, reproducible, and ready for regulatory review.

Frequently asked questions

What is the difference between training data and testing data?

Training data is the portion of a dataset a model learns from directly. During training, the algorithm sees both the input features and, in supervised learning, the correct labels, and it adjusts its parameters based on how well its predictions match those labels. Testing data is a separate portion, held out and never shown to the model during training, used purely to evaluate performance afterward. Keeping the two separate matters because a model can achieve a low error rate on training data simply by memorizing it, a failure mode called overfitting. Testing data reveals whether the model generalizes to unseen data or whether it has just memorized noise. Most workflows add a third split, validation data, used during training to tune hyperparameters and make decisions about the model without touching the test set. Only after those decisions are finalized does the test set get used, exactly once, to report an honest performance estimate. Reusing the test set to make further adjustments quietly turns it into another validation set and undermines the whole point of holding it out.

What is the difference between a model and an algorithm?

An algorithm is the general method or set of mathematical rules used to learn from data, for example logistic regression, a random forest, or a neural network architecture. It is reusable and untrained. The same algorithm can be applied to many different problems and datasets. A model is the specific output that results after that algorithm has been trained on a particular dataset. It is the algorithm plus the learned parameters, the specific weights, coefficients, or split points, that came from fitting it to your data. Two data scientists using the identical algorithm on different training data will end up with two different models, even though the underlying method is the same. This distinction matters in conversations about reproducibility. Saying you are deploying "the algorithm" is imprecise, because what actually runs in production is a trained model, a fixed set of parameters derived from a specific training run on a specific dataset at a specific point in time.

What is overfitting and how do you prevent it?

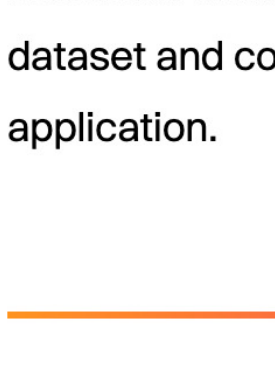
Overfitting happens when a model learns the training data too well, including its noise and quirks, rather than the underlying pattern that would generalize to new data. An overfit model shows excellent accuracy on training data but performs noticeably worse on a held-out test set, a gap that is usually the clearest sign something is wrong. Several techniques reduce overfitting. Cross-validation gives a more honest estimate of how a model will perform outside its training data and helps catch overfitting during development rather than after deployment. Regularization techniques add a penalty for excessive model complexity, discouraging the model from relying too heavily on any single feature. Simplifying the model, using fewer features or a less complex algorithm, also reduces the model's capacity to memorize noise. Finally, more training data, when available, gives the model a broader picture of the true pattern and makes memorization less likely to help. Most training pipelines apply a combination of these rather than relying on just one.

How long does it take to train a machine learning model?

Training time depends heavily on the size of the dataset, the complexity of the algorithm, and the compute available. A logistic regression model on a dataset with a few thousand rows might train in seconds on a laptop. A gradient boosted tree ensemble on millions of rows, with hyperparameter tuning across dozens of configurations, can take hours. Deep learning models, and particularly fine-tuning or pretraining large language models, are on another scale entirely. Training runs can take days or weeks across multiple GPUs, depending on model size and dataset scale. The honest answer for any specific project is that training time is usually not the bottleneck. Data collection, cleaning, and feature engineering typically take longer than the training step itself, and hyperparameter tuning can multiply the training time several times over as a team searches for the best configuration.

What is the difference between training and fine-tuning?

Training, in the traditional sense, starts a model from randomly initialized parameters and teaches it a task entirely from the training data provided. Fine-tuning starts from a model that has already been trained, usually a large foundation model pretrained on a broad, general dataset, and adapts it to a narrower task using a smaller, more specific dataset. The mechanics are the same. Both use a loss function and gradient-based updates to adjust parameters. What differs is the starting point and the scale of data needed. Because a fine-tuned model already understands general patterns, for example general language structure in the case of an LLM, it typically needs far less task-specific data and far less compute than training an equivalent model from scratch. Fine-tuning has become the default approach for adapting generative AI models to enterprise use cases, since training a comparable model from zero would require a dataset and compute budget well beyond what most organizations can justify for a single application.



David Weedmark

David Weedmark is a published author who has worked as a project manager, software developer and as a network security consultant.

Domino

Who is Domino?

Domino Data Lab empowers the largest AI-driven enterprises to build and operate AI at scale. Domino's Enterprise AI Platform provides an integrated experience encompassing model development, MLOps, collaboration, and governance. With Domino, global enterprises can develop better medicines, grow more productive crops, develop more competitive products, and more. Founded in 2013, Domino is backed by Sequoia Capital, Coatue Management, NVIDIA, Snowflake, and other leading investors.

WATCH DEMO

Platform
AI infrastructure
Data management
AI workbench
MLOps
AI governance
FinOps
Pricing
Security & compliance
What's new

Solutions
INDUSTRIES
Life sciences
Finance
Public sector
Retail
Manufacturing
USE CASES
Generative AI
Cost-effective data science
Self-service data science
Model risk management
Cloud data science

Learn
Events
Blog
Podcast
Courses and certifications
Data Science Dictionary
Documentation
Support
Demo hub

Company
About
Why Domino
Careers
News and press
Partners
Customers
Contact us

© 2026 Domino Data Lab, Inc. Made in San Francisco.